

What is an Anti-Pattern in Software Engineering?

An anti-pattern is:

- a **commonly repeated solution**
- to a **recurring problem**
- that turns out to be **ineffective, hard to maintain, or harmful**

They're the opposite of good design patterns like Singleton, Factory, Observer, MVC, etc.

A. Common Software Design Anti-Patterns

1. God Object / God Class

One class does everything.

Symptoms

- Huge class
- Too many responsibilities
- Hard to test
- Hard to reuse

2. Spaghetti Code

Code with tangled logic and dependencies.

Symptoms

- Deep nesting
- Random function calls
- Difficult debugging

3. Copy-Paste Programming

Duplicating logic everywhere.

Problem

- Bugs multiply
- Updates become painful

4. Golden Hammer

Using one technology or pattern for everything.

Example

“Everything should be microservices.”

Problem

Wrong tool for the job.

5. Lava Flow

Old unused code remains because nobody wants to remove it.

Symptoms

- Dead code
- Legacy modules
- Fear of deletion

6. Reinventing the Wheel

Building something that already exists and is proven.

Example

Creating a custom authentication system unnecessarily.

7. Mushroom Management

A management anti-pattern where developers are:

- kept in the dark,
- given little context,
- and only told what to do.

Symptoms

- No visibility into business goals
- Decisions made secretly
- Developers excluded from architecture discussions
- Communication flows only top-down
- Teams don't understand *why* they are building features

8. Input Kludge

An anti-pattern where invalid, messy, or poorly designed input handling is “patched” with hacks instead of proper validation/design.

Symptoms

- Endless input exceptions
- Strange parsing logic
- Multiple “special cases”
- Data cleanup everywhere
- Reliance on user behavior assumptions

B. How to Avoid Anti-Patterns

- Follow SOLID principles
- Follow OOP concepts
- Keep responsibilities small
- Refactor continuously
- Write tests
- Prefer simplicity
- Review architecture regularly
- Use patterns only when necessary

C. Difference: Pattern vs Anti-Pattern

Pattern	Anti-Pattern
Proven good solution	Common bad solution
Improves maintainability	Creates technical debt
Encourages scalability	Causes rigidity
Easier collaboration	Confusing codebas